Bach4Popper: Towards Federated Inductive Logic Programming using Coordination

Yasmine Akaichi[1 [0009−0005−5078−1583]],
Manel Barkallah[1 [0000−0003−2608−5658]],
Jean-Marie Jacquet[1 [0000−0001−9531−0519]],
Isabelle Linden[2 [0000−0001−8034−1857]], and
Wim Vanhoof[1 [0000−0003−3769−6294]]

[1] Nadi Research Institute, Faculty of Computer Science, University of Namur
Rue Grandgagnage 21, 5000 Namur, Belgium
{yasmine.akaichi,manel.barkallah}@unamur.be
{jean-marie.jacquet,wim.vanhoof}@unamur.be

[2] Nadi Research Institute, EMCP Faculty, University of Namur
Rempart de la Vierge 8, 5000 Namur, Belgium
isabelle.linden@unamur.be

Abstract. Although it is ubiquitous, machine learning still faces two major challenges: explainability and privacy. To address the first challenge, this paper focuses on Inductive Logic Programming (ILP), a framework for learning declarative, symbolic, and interpretable models from multi-relational data. However, existing ILP systems are fundamentally centralized, an assumption that conflicts with privacy requirements and with many real-world scenarios in which data are inherently distributed and cannot be shared. To overcome this limitation and address the second challenge, we introduce Bach4Popper, a federated ILP framework that combines Popper, an ILP learning system, with Bach, a coordination language developed by the authors. We show, both theoretically and empirically, that Bach4Popper is correct with respect to the corresponding centralized version of Popper. Experimental results further demonstrate that computational performance is preserved when moving from a centralized to a federated setting.

Keywords: Coordination · Federated Learning · Machine Learning · Inductive Logic Programming.

1 Introduction

Driven by the availability of large datasets and powerful computational resources, machine learning has become ubiquitous across a wide range of domains, including healthcare, finance, autonomous systems, and scientific discovery. Despite its success, machine learning faces two major and persistent challenges: explainability and privacy. Modern models, particularly deep neural networks,

often operate as black boxes, making their decisions difficult to interpret or justify, which limits trust, accountability, and adoption in high-stakes applications. This lack of transparency has motivated a growing body of work on explainable and interpretable machine learning. At the same time, machine learning systems are increasingly trained on sensitive personal or proprietary data, raising serious privacy concerns. Moreover, centralized data collection and training can expose individuals and organizations to risks such as data leakage, membership inference, and model inversion attacks, and may violate regulatory constraints such as data protection laws. Addressing explainability and privacy is therefore essential for the responsible and sustainable deployment of machine learning systems.

Being rooted in logic programming, which deals with symbolic programs of a declarative nature, inductive logic programming [11,24] provides an interesting solution to the explainability issue in machine learning. Indeed, inductive logic programming learns hypotheses expressed as sets of logical rules and thus makes explicit the relational structure and reasoning steps underlying predictions. As a result, this declarative and symbolic representation allows learned models to be directly inspected, validated, and reasoned about by domain experts.

However, inductive logic programming (ILP) is often formulated as a centralized learning task: it requires access to all data and assumes that the required knowledge, examples and background knowledge, is available on a single machine. This restricts its applicability in privacy-sensitive or multi-institutional settings where data must remain distributed, precisely in scenarios in which explainability offered by ILP would be most beneficial. Federated learning [23] has been designed precisely to that end. In fact, it provides a decentralized architecture in which a central server orchestrates training while data remain strictly local. However, most existing research on federated learning relies on sub-symbolic models, most often on neural networks, that offer limited explainability. Hence the question naturally arises whether inductive logic programming can be reformulated in a federated setting thereby offering a machine learning framework both explainable and respecting privacy and distributed issues.

The aim of this paper is to establish that data-driven coordination languages constitute a good means to combine federated learning and inductive logic programming. However, our work should be understood as a first step towards exploring the use of coordination languages for federated inductive logic programming. Rather than aiming for an exhaustive comparison of distributed computing paradigms, we focus on Bach, a Linda-like coordination language, as a representative and well-understood model that enables us to explicitly capture and analyze the coordination mechanisms underlying distributed hypothesis evaluation.

Nevertherless, in contrast to standard federated learning approaches, which typically rely on aggregating numerical model updates, our coordination-based approach enables the explicit orchestration of symbolic interactions, where only abstract evaluation outcomes are exchanged instead of data or model parameters.

On the learning side, we consider Popper [9], a recent ILP system that is particularly appealing due to its ability to learn from failures and to find glob-

ally optimal hypotheses. Bach [10], grounded in logic programming principles, provides the coordination layer through which distributed evaluation is orchestrated. The resulting combination, named Bach4Popper, is shown to be correct with respect to Popper, in the sense that it computes equivalent models. This is established both theoretically and experimentally, and we further show that computational performance is preserved when moving from a centralized to a federated setting.

The rest of the paper is organized as follows. The two following sections introduce the background knowledge in the two domains connected by our contribution. First, Section 2 introduces the basic notions on which *logic programming* relies and explains *inductive logic programming* with a particular focus on Popper. Then, Section 3 presents the main features of *the Bach coordination language* and details a socket-based implementation in Python, named BLPy. Using that background, Section 4 presents our main contributions : it introduces *the Bach4Popper framework* and *proves that it is correct* with respect to Popper. These theoretical concerns are completed with experimental ones in Section 5. It shows that these results are confirmed on experiments and demonstrates that computational performance is preserved when moving from a centralized to a federated setting. Section 6 compares our work to related work and Section 7 draws our conclusions.

2 Inductive Logic Programming

Inductive Logic Programming lies at the intersection of machine learning and logic programming. In brief, it aims to learn logical rules from positive and negative examples, as well as from background knowledge. All of these elements, rules, examples, and background knowledge are expressed within the framework of logic programming.

2.1 Logic Programming

A full introduction to the concepts underlying logic programming is outside the scope of the current paper. We assume the reader is familiar with these concepts, in particular with the notions of term and predicate, and refer him to [21] for a comprehensive overview. In this section, we will just mention that programs we shall consider consist of implications (or Horn clauses) of the form $\forall X_1 \cdots \forall X_n \; H \Leftarrow B_1 \wedge \cdots \vee B_m$ and that two semantics are given to them.

On the one hand, the declarative semantics of logic programs defines their meaning purely in logical terms without any reference to an execution model. Central to this semantics is the notion of logical consequence, which is defined by stating that a goal G is a logical consequence of a set of Horn clauses S iff every model of S is also a model of G. This is subsequently denoted as $S \models G$.

A major result of model theory in the case of Horn clauses is that the set M_P of logical consequences of a set of Horn clauses P can be determined as the intersection of all its so-called Herbrand models and may be further characterized as

the least fixed point of the immediate consequence operator T_P. Obviously, by construction, one has $T_P(M_P) = M_P$. It turns out that this result can be generalized for Horn clauses extended with negative literals in their body. We refer the reader to [3] for more explanations. For this introduction, it is sufficient to say that, under some conditions, the stable semantics developed by M. Gelfond and V. Lifschitz has allowed to identify stable models in the sense that they are fixed points of (suitable extensions) of the T_P operator dealing with negation. This has led to the development of so called answer set programming [20], which is a framework in which one computes stable models for logic programs, potentially extended to include constraints and optimization.

On the other hand, the operational semantics defines the semantics of Horn clauses through the successive derivation of a goal, each derivation producing a computed answer substitution, i.e. a single substitution summarizing the values given to the variables along the derivation path. In case the computation is successful, we shall use the notation $P \vdash G\ [\sigma]$ denote the existence of this successful derivation of the goal G yielding σ as a computed answer substitution.

The two semantics are established equivalent. See for instance [21] for a proof.

2.2 Inductive Logic Programming

Inductive Logic Programming (ILP for short) [25] aims to learn a hypothesis H describing a predicate as a logic program, using positive examples E^+ of this predicate, negative examples E^- of this predicate, and a background knowledge B_k, and this in a way such that $H \cup B_k$ entails all positive examples and none of the negative ones. ILP is typically applied in a centralized setting in which all examples are available on a single machine.

Several concepts need to be introduced to further characterize ILP. The background knowledge B_k is used to provide a context for learning. It contains a set B of facts and rules that describe relationships already understood about the domain. It also includes a so-called declaration bias D, which aims at restricting the hypotheses that can be constructed. Examples include predicate declarations, which specify which predicate names can appear in the head of clauses, in particular the form of the arguments, as well as body declarations, which do the same for predicates appearing in the body of clauses. It is assumed that no predicate symbol in the body of a clause in B_k appears in a head declaration of D. In other words, we assume that B_k does not depend on any hypothesis.

It is worth noting that predicate declarations can be expressed as constraints at the level of a meta-language. This is one of the core properties of the language Popper [9] we shall use. Assuming such a meta-language $\mathcal{L}$, a hypothesis constraint is a constraint expressed in $\mathcal{L}$. For instance, the atom `head_literal(Clause,Pred,Arity,Vars)` is used in Popper to denote that the clause `Clause` has a head literal with the predicate symbol `Pred`, is of arity `Arity`, and has the arguments `Vars`. As a further use

```
:- head_literal(_,p,2,_).
```

states that a predicate symbol `p` of arity 2 cannot appear in the head of any clause in a hypothesis.

The input of the inductive learning problem can then be formulated as composed of the following ingredients.

Definition 1. *The ILP problem input is a tuple* (B, D, C, E^+, E^-) *where*

- B *is a Horn program denoting background knowledge*
- D *is a declaration bias*
- C *is a set of hypothesis constraints*
- E^+ *is a set of ground atoms denoting positive examples of the predicate to be learnt*
- E^- *is a set of ground atoms denoting negative examples of the predicate to be learnt.*

We now define what constitutes a valid hypothesis for an ILP problem.

Definition 2. *Let* (B, D, C, E^+, E^-) *be an ILP problem input and* H *be a hypothesis. Then* H *is:*

- *complete when* $B \cup H \models e^+$ *for any* $e^+ \in E^+$
- *consistent when* $B \cup H \not\models e^-$ *for any* $e^- \in E^-$

A hypothesis is a solution when it is both complete and consistent.

Example 1. The *Zendo* dataset [5] is one of the benchmarks used ILP. The target predicate is `zendo/1`, whose argument is a state. The background knowledge B encodes properties and relations between objects. A fragment of B includes typically facts such as:

$B = \{$ `piece(s1,p1)`, `piece(s1,p2)`, `size(p1,1)`, `small(1)`, `red(p2)` $\}$

The declaration bias D constrains the hypothesis space by specifying admissible predicates and structural limits on candidate clauses. A simplified example of such a bias is:

$D = \{$`head_pred(zendo,1)`, `body_pred(piece,2)`, `max_clauses(4)`$\}$

Positive and negative examples are given as ground atoms over states:

$E^+ = \{$`zendo(s1)`, `zendo(s7)`$\}$ $\quad E^- = \{$`zendo(s2)`, `zendo(s6)`$\}$

The constraint set C is progressively refined during the learning process to prune the hypothesis space. For example, it may contain auxiliary rules and hard constraints such as:

```
included_clause(h,Cl) :- head_literal(Cl,zendo,1,(A)),
    body_literal(Cl,piece,2,(A,D)).
:- included_clause(h,C0), body_size(C0,5), C0 >= 0.
```

Here, `Cl` denotes a clause identifier used internally by Popper to represent candidate hypotheses. The auxiliary predicate `included_clause(h,Cl)` identifies clauses matching a given syntactic pattern, while the hard constraint excludes clauses of this form from the search space. This illustrates how C is refined to eliminate unpromising hypotheses.
An example of an explainable and human-readable hypothesis H learned on the *Zendo* dataset is:

$H = \{$`zendo(A):-piece(A,B),contact(B,D),size(D,C),small(C),red(B).` $\}$

This rule states that a state A satisfies the target concept `zendo/1` if it contains a piece B that is in contact with another piece D whose size is small, and where B is red.

In practice there can be many solutions to an inductive learning problem input. One is thus naturally interested by optimal solutions, which are typically defined by minimizing the number of literals used.

Definition 3. *The size of a hypothesis H, denoted $size(H)$, is the total number of literals employed in H. Given an inductive learning problem input, an optimal solution is defined as a solution H such that any other solution H' verifies $size(H) \leq size(H')$.*

It is not always possible to find a complete and consistent hypothesis. In that case, one tries to get the best hypothesis in the sense that it entails the largest number of positive examples and the smallest number of negative examples. This leads to the notions of confusion matrix and score.

Definition 4. *Let (B, D, C, E^+, E^-) be an input tuple and H be a hypothesis. Then the confusion matrix for H, denoted $conf_matrix(H)$, is the tuple (TP, FN, TN, FP) whose components are defined as follows:*

- *TP is the number of positive examples correctly entailed by $B \cup H$ (true positives)*
- *FN is the number of positive examples not entailed by $B \cup H$ (false negatives)*
- *TN is the number of negative examples not entailed by $B \cup H$ (true negatives)*
- *FP is the number of negative examples incorrectly entailed by $B \cup H$ (false positives)*

The score of H, denoted $score(H)$, is defined as $TP + TN$.

In the past thirty years, many inductive logic systems have been introduced. The reader is referred to [7] for a detailed description. Pioneer languages, such as Progol and Metagol, have explored subsumption lattices, amounting to trees of rules. In contrast, Popper has taken a meta-language approach which allows generating hypotheses by using Answer Set Programming (ASP) and which allows to learn from failures. It is generally considered as very appealing since it allows to find globally optimal hypotheses whereas Progol and Metagol may

produce only locally optimal hypotheses. We shall turn in this paper to Popper. Its main algorithm is described in Algorithm 1 as a Python function inspired by the one presented in [9]. The function takes as arguments the sets E^+, E^-, D, C described in Definition 1. Three other arguments are added to limit the search space: the maximum number of variables, literals and clauses in a hypothesis. The Python function returns a hypothesis: either a solution, if there is one, or an hypothesis that maximizes the score function otherwise. The algorithm mainly consists of two loops. The outer one (starting on line 9 of Algorithm 1) is responsible for searching hypotheses which increasingly add literals. For a fixed number of literals, the inner loop (starting on line 12) embodies the Popper generate-test-constraint loop. Given a current set `CC` of constraints, the call to the `generate` function (line 13) generates a hypothesis. This is obtained through Answer Set Programming acting over the constraints `CC` of a meta-language. The produced solution, if there is one, is a hypothesis meeting the constraints of `CC`. In case there is no solution then the variable `c_solvable` is set to false (line 15) which leads to finish the inner loop. Otherwise the solution is tested against the positive and negative examples by additionally using the background knowledge of `B`. This is operated by the call to the function `test` (line 17). This results in a confusion matrix as specified in Definition 4. Two outputs are computed therefrom: an `outcome` and a `score` (lines 18 and 19). The outcome summarizes how many positive and negative examples are covered. For the positive examples, either all of them are covered (which is denoted as "all"), or some are covered (which is denoted by "some") or none are covered (which is denoted by "none"). For the negative examples, either some are covered (which is denoted by "some", this including the case where all are covered) or none are covered (which is denoted by "none"). Obviously a solution is found in case (all,none) is produced. In the other cases, the article [9] explains which constraints are to be added to the `CC` constraints. Note that this is operated so as to exclude answer sets in the ASP resolution, which makes the algorithm converge. Technically in the algorithm this is achieved by the call to the function `learn_constraints` (line 26). The important point for our work is that the added constraints can be determined in view of the outcome and do not need the knowledge of positive nor negative examples. It is also worth noting that, having computed its score, the new hypothesis is retained as the best one if its score is better than the one memorized in variable `best_hyp` (line 25).

3 The BLPy coordination language

3.1 The Bach Coordination Language

Bach is a Linda-like coordination language developed at the University of Namur [15]. It has been conceived in a constraint logic programming flavor. As a result, Linda tuples are replaced by ground logic programming terms, also called si-terms, the shared space is referenced as the store, and Linda primitives are reformulated as the following primitives: $tell(t)$ to place an occurrence of t on

Algorithm 1 Popper (inspired by [9])

```
def popper(E+, E−, B, D, C, max_vars, max_literals,
    max_clauses):

  best_score = 0
  best_hyp = { }
  end_popper = false
  num_literals = 0
  CC = C

  while not end_popper and num_literals <= max_literals:
    c_solvable = true
    num_literals += 1
    while not end_popper and c_solvable:
      hyp = generate(D, CC, max_vars, num_literals,
                                          max_clauses)
      if not hyp:
        c_solvable = false
      else:
        conf_matrix = test(E+, E−, B, hyp)
        outcome = compute_outcome(conf_matrix)
        score = compute_score(conf_matrix)
        if outcome == ('all', 'none'):
          best_hyp = hyp
          end_popper = true
        else:
          if best_score < score:
            best_hyp = hyp
          CC += learn_constraints(hyp, outcome)

  return best_hyp
```

the store, $ask(t)$ to verify that t is present on the store, $get(t)$ to proceed similarly but by consuming one occurrence of t, and $nask(t)$ to check that t is absent from the store. Note that the last three primitives suspend until their associated condition is met.

Matching is operated in Bach as syntactic equality. We shall soften it in this paper by allowing a partially specified si-term to match a more complete si-term. This will be very useful later to get values from non specified arguments. The precise definition is as follows.

Definition 5. *Let $t = f(t_1, \cdots, t_m)$ and $u = g(u_1, \cdots, u_n)$ be two si-terms. Then t matches u iff, on the one hand, their functors are the same ie $f = g$ and, on the other hand, the list $[t_1, \cdots, t_m]$ of arguments of t is a prefix of the list $[u_1, \cdots, u_n]$ of arguments of u. This is subsequently denoted by $t \triangleleft u$. On the point of terminology, in that case, it is also said that u is matched by t.*

Primitives are composed in Bach to form more complex instructions, called agents, by using traditional composition operators from concurrency theory: sequential composition, parallel composition and non-deterministic choice. It is worth noting, however, that we shall only use the two first subsequently.

3.2 The BLPy implementation

For the purpose of exploiting federated inductive logic programming using Bach, an implementation has been developed in Python. It is based on sockets and organized around, on the one hand, a server, managing the store and the actions of primitives, and, on the other hand, clients interacting with the store. The core part is threefold: it consists in defining a grammar to parse the messages, in implementing the store, and in designing an interpreter. As regards the grammar, our work is based on the `parsimonious` module, which provides fast arbitrary-lookahead parsers written in pure Python. The store is implemented using three data structures:

- a dictionary, named `theStore`, which represents the real contents of the store. For each functor it contains as a key it associates a dictionary mapping si-terms formed with this functor to integers, denoting the numbers of occurrences of the considered si-terms currently on the store;
- a dictionary, named `theWaitingList`, that registers process identifiers waiting for the presence of si-terms. Technically, it associates with a functor name a list of triples, composed of a process identifier, the si-term on which the process is suspended and the destructive nature of the operation (according to the fact that the process is suspended on a ask or a get request);
- a dictionary, named `theNWaitingList`, that registers process identifiers waiting for the absence of si-terms. Its structure is similar to that of `theWaitingList` except that processes are there waiting for the absence of the corresponding si-terms and that no destructive feature is recorded.

That being said, the implementation of the primitives is as follows:

- the execution of $tell(t)$ consists in checking whether the functor of t, say f, is registered in `theStore`. If this is the case and if t already appears as a key of `theStore[f]` then the number of associated occurrences is increased by one. If f is a key of `theStore` but t is not a key of `theStore[f]` then a new entry is added for t with 1 as associated integer. If f is not a key of `theStore` then it is introduced in `theStore` keys with a mapping from t to 1 as associated value. In all the cases, the processes of `theWaitingList` are looked at to see whether processes can be waken up.
- the execution of $ask(t)$ is dual. It succeeds in case the functor f of t is a key of `theStore` and if there is a si-term matching t in `theStore[f]`. If this is not the case then the identifier of the process running $ask(t)$ together with t is added to `theWaitingList`.

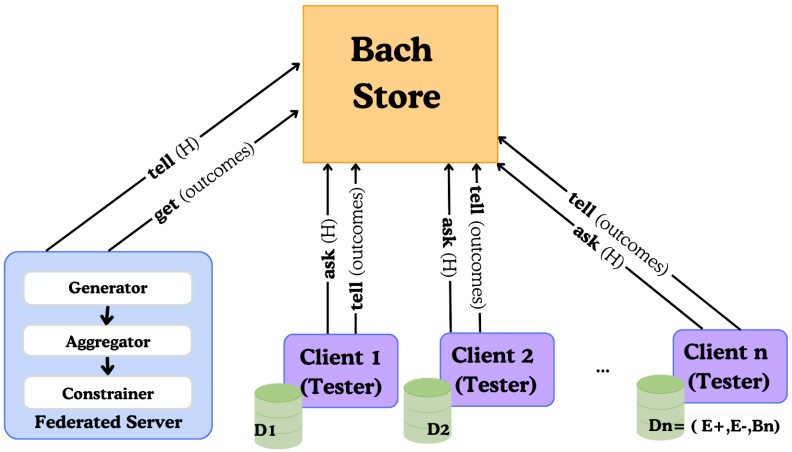

Fig. 1. Architecture of Bach4Popper illustrating the coordination mechanism underlying federated ILP. The federated server generates candidate hypotheses (Generator), aggregates client evaluations (Aggregator), and refines the hypothesis space through constraints (Constrainer). Hypotheses are communicated to clients via the shared Bach store using `tell`, while clients retrieve them using `ask`, evaluate them on local datasets, and return outcomes. This interaction implements a distributed version of the generate–test–constrain loop of Popper, where hypothesis evaluation is performed locally and coordination is handled through the shared store.

- the execution of $get(t)$ is similar but consists of removing an occurrence of a matching u is case such a si-term is present. In a dual perspective as $tell(t)$ processes of `theNWaitingList` are inspected to determine whether they can be waken up.
- finally the execution of $nask(t)$ is dual to that of $ask(t)$, asking for the absence of si-terms u matched by t and resulting in the process identifier being added to `theNWaitingList` together with t in case of suspension.

Sequential and parallel compositions are not implemented directly, but rather are handled at the Python level.

As a proof of concept, a Python package, named BLPy, has been developed. It can be downloaded at the address mentioned in [4].

4 The Bach4Popper Framework

4.1 The framework

We are now in a position to explain how to implement federated inductive logic programming using Popper and BLPy. As illustrated in Figure 1, given the

store made available by BLPy, the basic idea is to have a BLPy server process, named `bbpopper`, handling the store, and two types of BLPy client processes communicating through it. One of them, referred to as `srvpopper`, is deployed in a unique instance. It is responsible for the generate and constraint parts of Algorithm 1. The other types of clients, named `clipopper`, are deployed in as many instances as data entities available. They are responsible for the test part of Algorithm 1 on their local data. The two types of clients communicate via the store. It is assumed that the number of `clipopper` clients is known in advance, that `clipopper` clients are identified by numbers ranging from 1 to this number, and that no intruder modifies the content of the store.

Basically, from a coordination perspective, the `srvpopper` process iteratively publishes the current hypothesis to the shared store (through a tell primitive) and queries (by means of get primitives) the local evaluation feedback produced by the `clipopper` clients. Each client evaluates the hypothesis locally and returns the symbolic feedback derived from its confusion matrix. The `srvpopper` process aggregates these feedbacks, computes new logical constraints, and uses them to refine the hypothesis.

Algorithm 2 details the server-side learning procedure. It is directly derived from Algorithm 1. It uses the `srvpopper` function as the main function. The prototype is the same as the one of function `popper` in Algorithm 1 except that the argument B providing the background knowledge is omitted. This omission reflects the fact that background knowledge is locally available on the distributed `clipopper` processes. The initialization phase follows the same structure as in centralized Popper, with the addition of an integer variable `r` that tracks the current federated round in the generate–test–constrain cycle. The two loops in Algorithm 1 are also essentially repeated, with the following differences. Instead of calling the `test` function to determine the confusion matrix, the current hypothesis (in case it exists) is told via the execution of the `tell(hypsi(r,hyp))` primitive (see line 18). Then the `srvpopper` process waits for the confusion matrices computed by the `clipopper` processes by means of the execution of the `ask(fedcmtuple(r,i))`, the result being stored in an array `fedcm` (see line 20). Each element of this array reflects the result of a local computation, which is then aggregated by the `aggregate_outcome` and `aggregate_score` functions (see lines 21 and 22). The end of the execution is then similar to the one of function `popper`, with as two differences that the end of the execution is reported by telling an empty hypothesis by the primitive `tell(hypsi(r,end_hyp))` (see line 26) and that each iteration ends by updating the `r` counter (see line 31). The precise definitions of the `aggregate_outcome` and `aggregate_score` functions are given in Definition 9 when theoretical results will make clear what they should be.

A `clipopper` client, say identified by i, executes dually. Basically it iterates by asking for the current hypothesis, tests it on its local data E_i^+, E_i^-, B_i, computes the confusion matrix and tells it. This is operated by function `clipopper` of Algorithm 3.

Algorithm 2 Srvpopper client

```
def srvpopper(N, D, C, max_vars, max_literals, max_clauses):

    best_score = 0
    best_hyp = { }
    end_popper = false
    num_literals = 0
    CC = C
    r = 1

    while not end_popper and num_literals <= max_literals:
        c_solvable = true
        num_literals += 1
        while not end_popper and c_solvable:
            hyp = generate(D, CC, max_vars, num_literals,
                                                max_clauses)
            if not hyp:
                c_solvable = false
            else:
                tell(hypsi(r,hyp))
                for i in range(1,N+1):
                    fedcm[i] = ask(fedcmtuple(r,i))
                outcome = aggregate_outcome(fedcm)
                score = aggregate_score(fedcm)
                if outcome == ('all', 'none'):
                    best_hyp = hyp
                    end_popper = true
                    tell(hypsi(r,nil_hyp))
                else:
                    if best_score < score:
                        best_hyp = hyp
                        CC += learn_constraints(hyp, outcome)
             r = r+1

    return best_hyp
```

4.2 Theoretical results

As a summary of the previous subsection, by identifying the processes with their main function and by keeping the maximum number of variables, literals and clauses as local data of the Popper server, the federated computation consists in executing the following parallel agent:

$$\begin{aligned} FedPopper(N) = \; & bbpopper||srvpopper(N,D,C)|| \\ & clipopper(1,E_1^+,E_1^-,B_1)||...||clipopper(N,E_N^+,E_N^-,B_N) \end{aligned} \tag{1}$$

We now have to establish that the final hypothesis computed by this execution is the same as the one computed by function `popper` of Algorithm 1. This

Algorithm 3 Clipopper client

```
def clipopper(i, Ei+, Ei−, Bi):

        end_popper = false
        r = 1

        while not end_popper:
           hyp_asked = ask(hypsi(r))
           if (hyp_asked = hyp(r,end_hyp):
              end_popper = true
           else:
              conf_matrix = test(Ei+, Ei−, Bi, hyp_asked)
              tell(fedcmtuple(r,Id,conf_matrix)
           r = r+1
```

is obviously only possible if a relation is identified between positive and negative examples, on the one hand, and the background knowledge, on the other hand. This is precisely the aim of the following definitions.

Definition 6. *Let E be a set of ground atoms. The set $U(E)$ is defined as the set of ground terms formed from constants and functions appearing in the arguments of atoms of E. By extension, if E^+ and E^- are sets of positive and negative examples on a predicate p, the set $U(E^+, E^-)$ is defined as $U(E^+) \cup U(E^-)$.*

Definition 7. *A Horn clause program B is said to be well-structured if it can be partitioned into a set SF of facts and a set SH of definite Horn clauses whose literal arguments are variables and whose head does not involve predicate of SF. Such a partition is unique by construction. The two sets are respectively referred to as $sf(B)$ and $sh(B)$, respectively.*

Intuitively the federated learning we are thinking about involves background knowledge that is shared among distributed clients and data that is distinct on each client. Technically, this leads in the subsequent definition to define a partition as having the sets $sf(B_1)$, ..., $sf(B_n)$ form a partition of $sf(B)$ in the mathematical sense while the sets $sh(B_1)$, ..., $sh(B_n)$ and $sh(B)$ are identical.

Definition 8. *Let $E^+, E_1^+, \cdots, E_n^+$ and $E^-, E_1^-, \cdots, E_n^-$ be respectively sets of positive and negative examples on a predicate p. Let $B, B_1, \cdots, B_n$ be well-structured Horn clause programs. The set $\{(B_1, E_1^+, E_1^-), \cdots, (B_n, E_n^+, E_n^-)\}$ is a federated partition of (B, E^+, E^-) iff the following conditions hold:*

- *$U(sf(B_i)) \subseteq U(E_i^+, E_i^-)$, for any $i = 1, \cdots, n$*
- *$sf(B_1) \cdots, sf(B_n)$ form a partition of $sf(B)$*
- *the sets $U(E_i^+, E_i^-)$ and $U(E_j^+, E_j^-)$ are disjoint, for any $i, j = 1, \cdots, n$ with $i \neq j$*
- *one has $sh(B_1) = \cdots = sh(B_n) = sh(B)$*

- $E_1^+, \cdots, E_n^+$ *form a partition of* E^+
- $E_1^-, \cdots, E_n^-$ *form a partition of* E^-

Proposition 1. *Let* $\{(B_1, E_1^+, E_1^-), \cdots, (B_n, E_n^+, E_n^-)\}$ *be a federated partition of* (B, E^+, E^-) *with positive and negative examples over the predicate* p*. Let* H *be a Horn clause program. Then for any* $e^+ \in E^+$*, one has* $B \cup H \vdash e^+ [\theta]$ *iff* $B_i \cup H \vdash e^+ [\theta]$ *for some* $i = 1, \cdots, n$*.*

Proof. Assume first that $B \cup H \vdash e^+ [\theta]$. Then there exists a derivation $Deriv$ from e^+ using clauses of $B \cup H$ ending with the empty clause and producing θ as computed answer substitution. This derivation uses clauses of H, $sh(B)$ and facts from $sf(B)$. As $E_1^+, \cdots, E_n^+$ form a partition of E^+ there exists one and only one i such that $e^+ \in E_i^+$. In view of the federated partition assumption, the set of the facts used in $Deriv$ are then included in $sf(B_i)$. As the Horn clauses of $sh(B_i)$ are those of $sh(B)$, it follows that the derivation $Deriv$ is also a derivation establishing that $B_i \cup H \vdash e^+ [\theta]$.

Assume now $B_i \cup H \vdash e^+ [\theta]$, for some i. There is thus a derivation from e^+ using clauses of $B_i \cup H$ ending with the empty clause and producing θ as computed answer substitution. As $sh(B_i) = sh(B)$ and $sf(B_i) \subseteq sf(B)$, this derivation is also a derivation using clauses of $B \cup H$. It follows that $B \cup H \vdash e^+ [\theta]$.

As a direct consequence, any $e^- \in E^-$ is not derivable from $B \cup H$ if it is not derivable from $B_i \cup H$ for any i. However, the federated partition property allows to go one step further.

Proposition 2. *Let* $\{(B_1, E_1^+, E_1^-), \cdots, (B_n, E_n^+, E_n^-)\}$ *be a federated partition of* (B, E^+, E^-) *with positive and negative examples over the predicate* p*. Let* H *be a Horn clause program. Let* e^- *be a negative example of* E^-*. It is thus a negative example of some* E_j^-*. Then* e^- *is not derivable from* $B \cup H$ *iff it is not derivable from* $B_j \cup H$*.*

Proof. Indeed, in view of the federated partition assumption, $U(sf(B_j)) \subseteq U(E_j^+, E_j^-)$ and the sets $U(E_k^+, E_k^-)$ are disjoint. Therefore checking whether $B \cup H \vdash e^+ [\theta]$ holds amount to checking whether $B_j \cup H \vdash e^+ [\theta]$ hold.

It follows from propositions 1 and 2 that the confusion matrix computed (centrally) on (B, E^+, E^-) consists of the component-based sum of the confusion matrices computed (locally) for the (B_i, E_i^+, E_i^-).

Proposition 3. *Let* $\{(B_1, E_1^+, E_1^-), \cdots, (B_n, E_n^+, E_n^-)\}$ *be a federated partition of* (B, E^+, E^-) *with positive and negative examples over the predicate* p*. Let* H *be a Horn clause program. Let* (TP, FN, TN, FP) *be the confusion matrix for* H *determined with respect to* (B, E^+, E^-)*. For any* $i = 1, \cdots, n$*, let* (TP_i, FN_i, TN_i, FP_i) *be the confusion matrix for* H *determined with respect to* (B_i, E_i^+, E_i^-)*. Then*

$$TP = \sum_{i=1}^{n} TP_i \qquad TN = \sum_{i=1}^{n} TN_i$$
$$FP = \sum_{i=1}^{n} FP_i \qquad FN = \sum_{i=1}^{n} FN_i$$

Proof. Direct consequence of Propositions 1 and 2

This result allows for a direct aggregation of confusion matrices. However as the confusion matrix is not used as such but through the outcome and score only, it is possible to go one step further. Before that, let us define an aggregation of outcomes and scores as follows.

Definition 9. *Define* $\oplus : \{all, some, none\} \times \{all, some, none\} \rightarrow \{all, some, none\}$ *as follows:*

$$o_1 \oplus o_2 = \begin{cases} all & \text{if } o_1 = o_2 = all \\ none & \text{if } o_1 = o_2 = none \\ some & \text{otherwise} \end{cases}$$

It is easy to verify that the operator $\oplus$ is commutative and associative. This allows to state the following proposition.

Proposition 4. *Let* $\{(B_1, E_1^+, E_1^-), \cdots, (B_n, E_n^+, E_n^-)\}$ *be a federated partition of* (B, E^+, E^-) *with positive and negative examples over the predicate* p*. Let* H *be a Horn clause program. Let* (po, no) *and* s *be respectively the outcome and score computed for* H *with respect to* (B, E^+, E^-)*. Moreover, for any* $i = 1, \cdots, n$*, let* (po_i, no_i) *and* s_i *be respectively the outcome and score computed for* H *with respect to* (B_i, E_i^+, E_i^-)*. Then*

$$po = po_1 \oplus \cdots \oplus po_n$$
$$no = no_1 \oplus \cdots \oplus no_n$$
$$s = s_1 + \cdots + s_n$$

Proof. The proposition directly follows from Proposition 3.

The correctness of the federated computation embodied in Equation 1 and Algorithms 2 and 3 with respect to the central computation embodied in Algorithm 1 then comes from the fact that hypotheses *hyp* successively computed by Algorithm 1 (see line 12) are those that are successively told in Algorithm 2 (see line 14). This is easily established by induction, as thanks to Proposition 4, the outcome and score computed in a distributed manner are exactly the same as those computed centrally and therefore induce identical constraints for the generation phase.

5 Experiments

In this section, we evaluate `Bach4Popper` from two complementary perspectives[3]:

- *P1 (Correctness).* Does `Bach4Popper` compute hypotheses that are equivalent to those learned by (centralized) Popper despite strict data locality constraints and the absence of shared examples?

[3] To allow the interested reader to reproduce the experiments, the code of Bach4Popper is available at the address mentioned in [1].

Dataset	Target Pred.	#Pos	#Neg	MaxClauses	MaxVars	MaxBody
`Trains`	`f/1`	5	5	3	5	6
`Zendo1`	`zendo/1`	20	20	4	5	7
`iggp-rps`	`next_score/1`	108	356	4	5	7

Table 1. Datasets and corresponding Popper configuration used in the experiments.

- *P2 (Performance and Coordination Cost).* How does coordination-based federated execution impact the symbolic search process in terms of execution time compared to (centralized) Popper?

The evaluation is conducted on standard ILP benchmark datasets obtained from the official Popper implementation [6]. These benchmarks are widely used to assess symbolic hypothesis search under controlled bias settings. Table 1 summarizes the datasets along with their corresponding configurations. Note that, as we do not have access to data really distributed, we shall partition these datasets in order to fulfill the hypotheses of Definition 8. Each client thus only accesses its own examples, while coordination is handled through a central server.

We consider three representative benchmarks covering different levels of relational complexity:

- *Trains.* The `Trains` dataset [17] is a classical ILP benchmark where the goal is to distinguish between eastbound and westbound trains. Although relatively small, it requires structured relational reasoning over background knowledge and is commonly used to validate the correctness of ILP systems.
- *Zendo.* The `Zendo` dataset [5] consists of structured configurations from which a hidden concept must be inferred. Inspired by a rule-discovery game, it features a richer hypothesis space than *Trains* and is therefore suitable for evaluating symbolic generalization.
- *IGGP-RPS.* The `IGGP-RPS` dataset originates from Inductive General Game Playing [8] and involves learning rules that describe state transitions in a game environment. This benchmark is more challenging due to both the size of the example set and the complexity of the relational background knowledge.

We consider a cross-silo federated learning setting composed of three clients and a central server. To simulate this setting, each dataset is partitioned into three client-specific subsets, each containing a balanced distribution of positive and negative examples. Each client evaluates candidate hypotheses using only its local data, while coordination is performed by the server. No examples, background knowledge, or intermediate evaluation results are shared between clients or with the server. The server has access only to the bias.

Setting	Trains		Zendo		IGGP-RPS	
	Accuracy	Recall	Accuracy	Recall	Accuracy	Recall
Centralized	1.00	1.00	1.00	1.00	1.00	1.00
Bach4Popper	1.00	1.00	1.00	1.00	1.00	1.00

Table 2. Convergence of Bach4Popper towards centralized Popper in terms of descriptive performance.

5.1 Correctness

Unlike statistical learning, Popper seeks a hypothesis that is logically complete and consistent with the specification. Thus, accuracy and recall here indicate logical satisfaction rather than predictive generalization: a value of 1.00 means that all positive examples are entailed and all negative examples are rejected.

We first evaluate whether Bach4Popper preserves the models computed by centralized Popper. To that end, we consider a hybrid evaluation protocol combining federated learning with centralized testing. Algorithm 2 is enriched with access to the full dataset: lines 16-18 of Algorithm 1 are inserted before line 18 of Algorithm 2, after which outcomes and scores obtained in the federated and centralized settings are compared.

As shown in Table 2, the coordinated learning process consistently converges to hypotheses that are complete and consistent on the full dataset, confirming that federation preserves the correctness guarantees, thereby addressing *P1 (Correctness)*. This provides an empirical validation of Propositions 1-4 established in Section 4, showing that federated evaluation preserves the logical correctness of the learned hypotheses.

5.2 Performance and Symbolic Cost

We now evaluate the execution cost induced by distributing the testing phase of Popper. Unlike statistical federated learning, Bach4Popper does not distribute model optimization but preserves the symbolic generate-test-constrain loop of Popper while delegating hypothesis evaluation to multiple clients. To analyze this cost, we measure the following:

- $tCentralPopper$: time spent inside the symbolic ILP engine (generation of hypotheses, constraint construction, grounding, and solver update);
- $tFedPopper$: time spent performing federated coordination, i.e., transmitting hypotheses, waiting for client evaluations, and aggregating outcomes;
- $global_time$: total execution time from the first generated hypothesis until termination.

Two timing modes are used. On the one hand, by using Python `process_time()`, we measure only CPU time and ignore waiting periods. This shows the pure computation cost of the algorithm. On the other hand, by using

Dataset	With Suspension			Without Suspension		
	CENT (s)	FED (s)	Glob (s)	CENT (s)	FED (s)	Glob (s)
`Trains`	0.1437	26.9676	27.2257	0.0415	0.0073	0.0790
`Zendo`	0.1682	5.3198	5.7002	0.2087	0.0260	0.5039
`IGGP-RPS`	268.77	100.91	600.04	275.83	2.7582	483.15

Table 3. Execution time decomposition of Bach4Popper. "With Suspension" uses `perf_counter()` (including communication delays), while "Without Suspension" uses `process_time()` (CPU time only). CENT denotes the Popper symbolic core and FED the federated coordination cost.

Python `perf_counter()`, we measure real elapsed time, including communication and synchronization between clients. In other words, the first mode shows how much computation federation adds, while the second shows the actual runtime in a distributed setting.

This distinction is clearly reflected in Table 3. For all datasets, the federated cost (FED) drops significantly when suspension is removed. For instance, on the *Trains* dataset, FED decreases from 26.96s to 0.0073s, and on *Zendo*, from 5.31s to 0.026s. Even for the more complex *IGGP-RPS* dataset, FED drops from 100.91s to only 2.75s.

This behavior is also influenced by the complexity of the learning task. For simpler datasets such as *Trains* and *Zendo*, the search converges in fewer iterations, and the execution time is therefore largely dominated by coordination latency. In contrast, for more complex datasets such as *IGGP-RPS*, a higher number of iterations is required, which leads to both increased computational cost and an accumulation of coordination delays.

This consistent behavior across datasets shows that the overhead introduced by federation is primarily due to coordination latency, i.e., communication, synchronization, and waiting between components, rather than actual computation. In particular, the very low FED cost in the CPU-only setting indicates that the distributed evaluation itself is computationally lightweight.

Overall, Bach4Popper preserves the computational behavior of Popper, and the additional cost of federation mainly originates from distributed execution rather than increased algorithmic complexity. These results address point *P2* above.

Although execution time is influenced by the size and complexity of the datasets, the results show that this factor alone does not account for the observed overhead. The discrepancy between CPU time and wall-clock time demonstrates that coordination introduces additional latency that is independent of the underlying ILP computation. This confirms that the cost of federation arises not only from larger problem instances, but also from the coordination mechanisms required to distribute the evaluation process.

6 Related Work

Federated learning has emerged as a powerful paradigm for collaborative model training without centralizing data, addressing privacy, regulatory, and data-silo constraints [18,30]. However, most approaches rely on sub-symbolic models such as deep neural networks, whose decision processes are inherently opaque. While numerous explainability techniques have been proposed, they are predominantly post-hoc, providing approximate explanations that are not guaranteed to faithfully reflect the underlying model behavior [22]. Our work contrasts by adopting from the outset the interpretable setting of inductive logic programming. This has allowed us to establish theoretical properties which have no counterpart in these pieces of work.

Beyond neural networks, several works have explored interpretable or symbolic models in federated settings, including decision trees [13,19,28] and fuzzy logic models [14,16,26,27,29]. These approaches demonstrate that interpretability can be integrated into federated learning, but they typically focus on aggregating model structures or parameters. In contrast to our work, they are not designed to support relational hypothesis induction under explicit background knowledge, as required in Inductive Logic Programming, and do not emphasize the logical semantics and soundness guarantees central to logic programming systems.

As regards coordination, to the best of our knowledge, the article [12] is the only other coordination-based approach that has coped with federated learning. However, the approach is quite different from ours. First, the authors have used aggregate-based computing whereas we have employed a classical Linda-like approach. Moreover, as their goal is to tackle heterogeneity in data distribution, they have proposed Field-Based Federated Learning to allow the construction of several personalized model zones, each with its own model tailored to its local data, instead of a single global model shared across all data entities. We adopt an opposite perspective in our work by addressing data entities of a similar nature and showing that a model that would have been classically learned in a centralized way can actually be learned in a distributed manner.

Finally, our work is a continuation of a first approach in federated learning in which we explored the aggregation of locally learned hypotheses through majority voting [2]. However, in contrast to this work, our approach operates at the level of complete hypotheses and does not federate the internal learning process itself.

7 Conclusion

In the aim of addressing two main concerns in current machine learning techniques – explainability and privacy – this paper has introduced a new framework for federated learning based on the Bach coordination language and the inductive logic programming system Popper. This framework, named Bach4Popper, consequently relies on coordination primitives to learn Horn clauses from failures

in a distributed manner while preserving data locality and thus without sharing examples.

Technically, the core contribution of this work lies in the federation of Popper's internal generate-test-constrain loop. Hypothesis generation and constraint induction are performed centrally, while hypothesis evaluation is delegated to distributed clients that operate exclusively on local data. Through symbolic aggregation of local outcomes and scores, the global learning process remains logically equivalent to centralized Popper.

We formally proved that, under well-defined federated partitioning assumptions, the symbolic outcomes and scores computed locally by distributed clients can be exactly aggregated to recover their centralized counterparts. As a result, Bach4Popper is correct with respect to centralized Popper in the sense that the same models are computed. Experimental results on standard ILP benchmarks confirm this analysis. Across all datasets, Bach4Popper converges to hypotheses that are descriptively equivalent to those learned centrally in terms of accuracy and recall. Moreover, experiments show that the computational performance of the federated setting is reasonably close to that of a centralized computation.

Future work will focus on exploring larger relational domains, studying heterogeneous and noisy data distributions, and extending the theoretical results of Section 5 to other forms of partitioning.

8 Acknowledgment

The authors thank the University of Namur for its support. They also thank the Walloon Region for partial support through the Ariac project (convention 210235) and the CyberExcellence project (convention 2110186).

References

1. Akaichi, Y., Jacquet, J.M.: The Bach4Popper Framework. https://doi.org/https://doi.org/10.5281/zenodo.18981901
2. Akaichi, Y., Jacquet, J.M., Linden, I., Vanhoof, W.: Federated Inductive Logic Programming for Explainable Artificial Intelligence. In: Proceedings of the Second Workshop on Explainable Artificial Intelligence for the medical domain-25-30 October (2025)
3. Apt, K.R., Bol, R.N.: Logic Programming and Negation: a Survey. Journal of Logic Programming **19/20**, 9–71 (1994)
4. Barkallah, M., Jacquet, J.M.: The BLPy Framework. https://doi.org/10.5281/zenodo.18672845
5. Bramley, N.R., Rothe, A., Tenenbaum, J.B., Xu, F., Gureckis, T.M.: Grounding Compositional Hypothesis Generation in Specific Instances. In: Proceedings of the Annual Meeting of the Cognitive Science Society. vol. 40 (2018)
6. Cropper, A.: Popper: An Inductive Logic Programming System. `https://github.com/logic-and-learning-lab/Popper` (2020)
7. Cropper, A., Dumančić, S.: Inductive Logic Programming at 30: a New Introduction. Journal of Artificial Intelligence Research **74**, 765–850 (2022)

8. Cropper, A., Evans, R., Law, M.: Inductive General Game Playing. Machine Learning **109**(7), 1393–1434 (2020)
9. Cropper, A., Morel, R.: Learning Programs by Learning from Failures. Machine Learning **110**(4), 801–856 (2021)
10. Darquennes, D., Jacquet, J.M., Linden, I.: On Multiplicities in Tuple-based Coordination Languages: The Bach Family of Languages and its Expressiveness Study. Lecture Notes in Computer Science **10852 LNCS**, 81 – 109 (2018)
11. De Raedt, L.: Logical and Relational Learning. Springer Science & Business Media (2008)
12. Domini, D., Aguzzi, G., Esterle, L., Viroli, M.: Field-Based Coordination for Federated Learning. In: Castellani, I., Tiezzi, F. (eds.) Proceedings of the 26th International Conference on Coordination Models and Languages. Lecture Notes in Computer Science, vol. 14676, pp. 56–74. Springer (2024)
13. Dong, T., Li, S., Qiu, H., Lu, J.: An Interpretable Federated Learning-based Network Intrusion Detection Framework. arXiv preprint arXiv:2201.03134 (2022)
14. Ducange, P., Marcelloni, F., Renda, A., Ruffini, F.: Federated Learning of XAI Models in Healthcare: a Case Study on Parkinson's Disease. Cognitive Computation **16**(6), 3051–3076 (2024)
15. Jacquet, J.M., Linden, I.: Coordinating Context-Aware Applications in Mobile Ad Hoc Networks. In: Proceedings of the first ERCIM workshop on eMobility. pp. 107–118 (2007)
16. Jiang, W., Zhang, Y., Han, H., Liu, X., Gwak, J., Gu, W., Shankar, A., Maple, C.: Fuzzy Ensemble-based Federated Learning for EEG-based Emotion Recognition in Internet of Medical Things. Journal of Industrial Information Integration **44**, 100789 (2025)
17. Larson, J., Michalski, R.S.: Inductive Inference of VL Decision Rules. ACM SIGART Bulletin **1**(63), 38–44 (1977)
18. Li, Q., Wen, Z., Wu, Z., Hu, S., Wang, N., Li, Y., Liu, X., He, B.: A Survey on Federated Learning Systems: Vision, Hype and Reality for Data Privacy and Protection. IEEE Transactions on Knowledge and Data Engineering **35**(4), 3347–3366 (2021)
19. Li, Q., Zhaomin, W., Cai, Y., Yung, C.M., Fu, T., He, B., et al.: Fedtree: a Federated Learning System for Trees. Proceedings of Machine Learning and Systems **5** (2023)
20. Lifschitz, V.: Answer Set Programming and Plan Generation. Artificial Intelligence **138**(1-2), 39–54 (2002)
21. Lloyd, J.W.: Foundations of Logic Programming, 2nd Edition. Springer (1987)
22. Lopez-Ramos, L.M., Leiser, F., Rastogi, A., Hicks, S., Strümke, I., Madai, V.I., Budig, T., Sunyaev, A., Hilbert, A.: Interplay between Federated Learning and Explainable Artificial Intelligence: a Scoping Review. arXiv preprint arXiv:2411.05874 (2024)
23. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient Learning of Deep Networks from Decentralized Data. In: Artificial intelligence and statistics. pp. 1273–1282. PMLR (2017)
24. Muggleton, S.: Inductive Logic Programming. New generation computing **8**(4), 295–318 (1991)
25. Muggleton, S.: Inverse Entailment and Progol. New generation computing **13**(3), 245–286 (1995)
26. Połap, D.: Fuzzy Consensus with Federated Learning Method in Medical Systems. IEEE Access **9**, 150383–150392 (2021)

27. Srivastava, V., Lamba, V., Mathada, V.S., Bulla, C., Gupta, N., Veeramanikandan, P., et al.: An IoT-based Framework Employing Fuzzy Logic and Federated Learning for Decentralized Decision-making. International Journal of Information Technology pp. 1–7 (2025)
28. Truex, S., Baracaldo, N., Anwar, A., Steinke, T., Ludwig, H., Zhang, R., Zhou, Y.: A Hybrid Approach to Privacy-preserving Federated Learning. In: Proceedings of the 12th ACM workshop on artificial intelligence and security. pp. 1–11 (2019)
29. Wilbik, A., Grefen, P.: Towards a Federated Fuzzy Learning System. In: 2021 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE). pp. 1–6. IEEE (2021)
30. Yang, Q., Liu, Y., Chen, T., Tong, Y.: Federated Machine Learning: Concept and Applications. ACM Transactions on Intelligent Systems and Technology (TIST) **10**(2), 1–19 (2019)